

9

Structural Patterns

This chapter explores four design patterns from the well-known **Gang of Four (GoF)**. We use structural patterns to create complex, flexible, and fine-grained classes.

Structural patterns help us build and organize complex object hierarchies in a maintainable fashion. They allow us to add behaviors to existing classes dynamically, whether we designed the initial system this way or as an afterthought that emerges out of necessity later in the program's lifecycle.

The following topics will be covered in this chapter:

- Implementing the Decorator design pattern
- Implementing the Composite design pattern
- Implementing the Adapter design pattern
- Implementing the Façade design pattern

The first two patterns help us extend a class dynamically and efficiently manage a complex object structure. The last two help us adapt an interface to another or shield a complex system with a simple interface. Let's begin!

Implementing the Decorator design pattern

The Decorator pattern allows us to extend objects at runtime while separating responsibilities. It is a simple but powerful pattern. In this section, we explore how to implement this pattern in the traditional way and how to leverage an open source tool named **Scrutor** to help us create powerful dependency injection-ready decorators using .NET.

Goal

The decorator's goal is to extend an existing object, at runtime, without changing its code. Moreover, the decorated object should not be aware that it is being decorated, leaving it as a great candidate for long-lived or complex systems that need to evolve. This pattern fits systems of all sizes.

I often use this pattern to add flexibility and create adaptability to a program for next to no cost. In addition, small classes are easier to test, so the Decorator pattern adds ease of testability into the mix, making it worth mastering.

The Decorator pattern makes it easier to encapsulate responsibilities into multiple classes, instead of packing multiple responsibilities inside a single class. Having multiple classes that each holds a single responsibility makes the system easier to manage.

Design

A **decorator** class must both implement and use the interface that's being implemented by the **decorated** class. Let's see this step by step, starting

with a non-decorated class design:

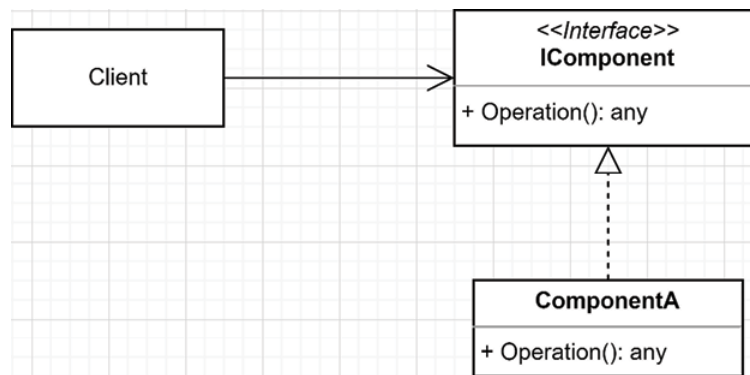


Figure 9.1: A class diagram representing the ComponentA class implementing the IComponent interface

In the preceding diagram, we have the following components:

- A client that calls the `Operation()` method of `IComponent`.
- `ComponentA`, which implements the `IComponent` interface.

This translates into the following sequence diagram:

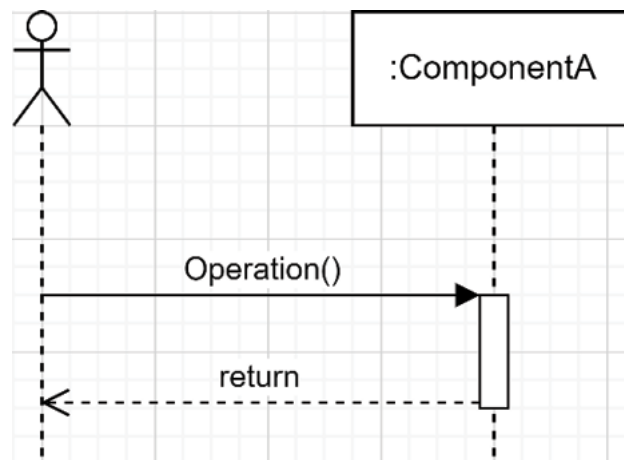


Figure 9.2: A sequence diagram showing a consumer calling the Operation method of the ComponentA class

Now, say that we want to add some new behavior to `ComponentA`, but only in some cases. In other cases, we want to keep the initial behavior. To do so, we could choose the Decorator pattern and implement it as follows:

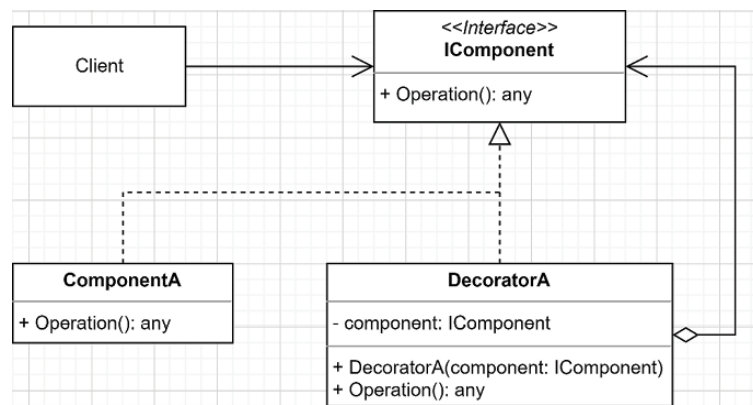


Figure 9.3: Decorator class diagram

Instead of modifying the `ComponentA` class, we created `DecoratorA`, which implements `IComponent` as well. This way, `Client` can use an instance of `DecoratorA` instead of `ComponentA` and have access to the new behavior, without impacting the other consumers of `ComponentA`. Then, to avoid rewriting the whole component, an implementation of `IComponent` is injected when creating a new `DecoratorA` instance (constructor injection). This new instance is stored in the `component` field and used by the `Operation()` method (implicit use of the **Strategy** pattern).

We can translate the updated sequence like so:

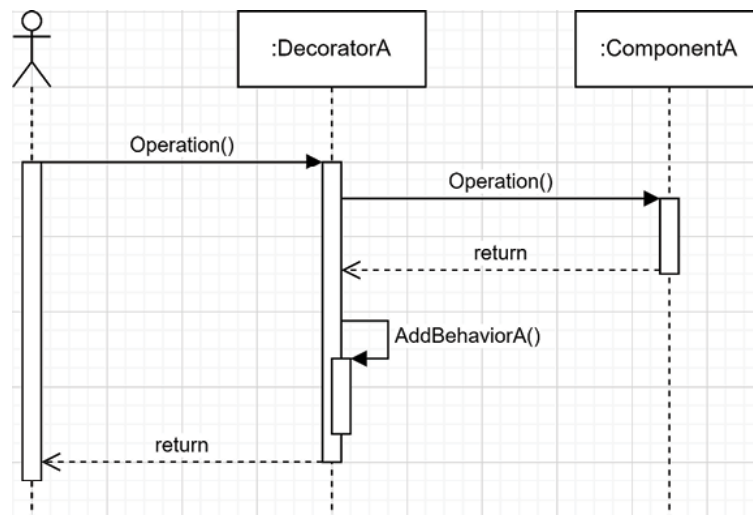


Figure 9.4: Decorator sequence diagram

In the preceding diagram, instead of calling `ComponentA` directly, `Client` calls `DecoratorA`, which in turn calls `ComponentA`. Finally, `DecoratorA` does some postprocessing by calling its private method; that is, `AddBehaviorA()`.

Note

Nothing from the Decorator pattern limits us from doing pre-processing, postprocessing, wrapping the decorated class's call (the `Operation` method in this example) with some logic (like an `if` statement or a `try-catch`), or all of that combined.

To show you how powerful the Decorator pattern is before we jump into the code, know this: we can chain decorators! Since our decorator depends on the interface (not the implementation), we could inject another decorator, say `DecoratorB`, inside `DecoratorA` (or vice versa). We could then create an infinite chain of rules that decorate one another, leading to a very powerful yet simple design.

Let's take a look at the following class diagram, which represents our chaining example:

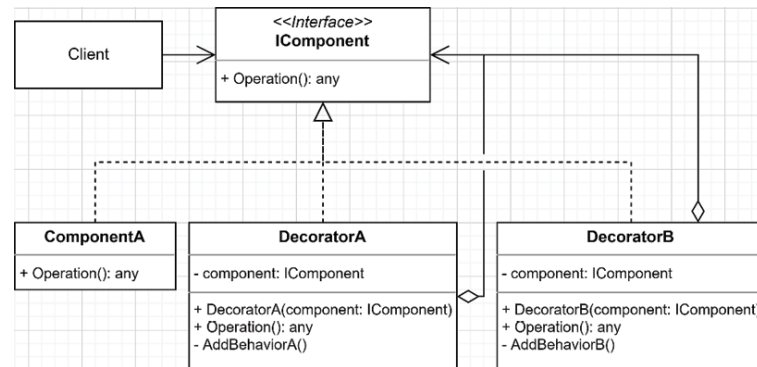


Figure 9.5: Decorator class diagram, including two decorators

Here, we created the `DecoratorB` class, which looks very similar to `DecoratorA` but has a private `AddBehaviorB()` method instead of `AddBehaviorA()`.

Note

The way we implement the decorator's changes in behavior or state is irrelevant to the pattern, which is why I excluded the `AddBehaviorA()` method from the initial class diagram: to show you only the pattern. However, I added it to this one (Figure 9.5) to make the idea behind having a second decorator clearer.

Let's take a look at the sequence diagram for this:

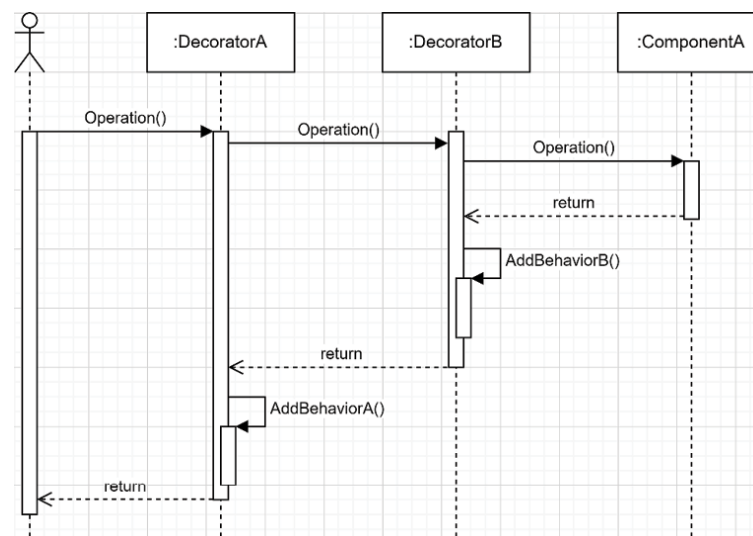


Figure 9.6: Sequence diagram of two nested decorators

With this, we are beginning to see the power of decorators. In the preceding diagram, we can assess that the behaviors of `ComponentA` have been changed twice without `Client` knowing about it. All those classes are unaware of the next `IComponent` in the chain. They don't even know that they are being decorated. They only play their role in the plan—that's all.

It is also important to note that the decorator's power resides in its dependency on the interface, not on an implementation, making it reusable. Based on that fact, we could swap `DecoratorA` and `DecoratorB` to invert the order the new behaviors are applied without touching the code itself. We could also apply the same decorator (say `DecoratorC`) to multiple `IComponent` implementations, like decorating both `DecoratorA` and `DecoratorB`.

Project – Adding behaviors

To help visualize the Decorator pattern, let's implement the previous example, which adds some arbitrary behaviors. Each `Operation()` method returns a string that is then outputted to the response stream. It is not fancy, but it shows how this works.

First, let's look at the `IComponent` interface and the `ComponentA` class:

```
public interface IComponent
{
    string Operation();
}
public class ComponentA : IComponent
{
    public string Operation()
    {
        return "Hello from ComponentA";
    }
}
```

The `IComponent` interface only states that an implementation should have an `Operation()` method that returns a `string`. The `Operation()` method implementation of the `ComponentA` class returns a literal string.

Now that we described the first pieces, let's look at the consumer, which is a minimal API endpoint:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<IComponent, ComponentA>();
var app = builder.Build();
app.MapGet("/", (IComponent component) => component.Operation());
app.Run();
```

In the `Program.cs` file above, we register `ComponentA` as the implementation of `IComponent`, with a singleton lifetime. We then inject an `IComponent` implementation when an HTTP request hits the `/` endpoint. The delegate then calls the `Operation()` method (see highlight).

At this point, running the application (from any URL) will result in the following output:

■

Hello from ComponentA

This happened because the system injected an implementation of the `IComponent` interface of type `ComponentA` into the `/` delegate, so the client outputs the result of `ComponentA`.

Next, we add the first decorator.

DecoratorA

Here, we want to modify the response without touching `ComponentA`. To do so, let's create a decorator named `DecoratorA` that wraps the `Operation()` result into a `<DecoratorA>` tag:

```
public class DecoratorA : IComponent
{
    private readonly IComponent _component;
    public DecoratorA(IComponent component)
    {
        _component = component ?? throw new ArgumentNullException(nameof(component));
    }
    public string Operation()
    {
        var result = _component.Operation();
        return $"<DecoratorA>{result}</DecoratorA>";
    }
}
```

`DecoratorA` depends on an implementation of `IComponent`. It uses that `IComponent` in the `Operation()` method and wraps its result in an HTML-like (XML) tag.

Now that we have a decorator, we need to tell the IoC container to send an instance of `DecoratorA` instead of `ComponentA` when injecting an `IComponent` interface.

`DecoratorA` should decorate `ComponentA`; or, more precisely, `ComponentA` should be injected into `DecoratorA`.

To achieve this, we could register it as follows:

```
builder.Services.AddSingleton<IComponent>(serviceProvider => new DecoratorA(new ComponentA()));
```

Here, we are telling ASP.NET Core to inject an instance of `DecoratorA` that decorates an instance of `ComponentA` when injecting an `IComponent` interface. When we run the application, we should see the following result in the browser:

`<DecoratorA>Hello from ComponentA</DecoratorA>`

Note

You may have noticed a few `new` keywords there, but even though it is not very elegant, we can manually create new instances in the composition root without jeopardizing our application's health. We learn how to get rid of some of them later with the introduction of `Scrutor`.

Next, we create a second decorator.

DecoratorB

Now that we have a decorator, it is time to create a second decorator to demonstrate the power of chaining decorators.

Context: At some point, we need to wrap that content once again, but we don't want to modify any existing classes. To achieve this, we concluded that creating a second decorator would be perfect, so we created the following `DecoratorB` class:

```
public class DecoratorB : IComponent
{
    private readonly IComponent _component;
    public DecoratorB(IComponent component)
    {
        _component = component ?? throw new ArgumentNullException(nameof(component));
    }
    public string Operation()
    {
        var result = _component.Operation();
        return $"<DecoratorB>{result}</DecoratorB>";
    }
}
```

It is very similar to `DecoratorA`, but the HTML-like tag is `DecoratorB` instead. The important and similar part here is that we always depend on the `IComponent` abstraction, never on any concrete class. This is what gives us the flexibility of decorating any `IComponent`, and this is what enables us to chain decorators.

To complete this example, we need to update our composition root like this:

```
builder.Services.AddSingleton<IComponent>(<serviceProvider => new DecoratorB(new DecoratorA(builder.Services.GetSingleton<IComponent>())
```

Now, we can decorate `DecoratorA` with `DecoratorB`, which, in turn, decorates `ComponentA`. Upon running the application, you should see the following output:

```
<DecoratorB><DecoratorA>Hello from ComponentA</DecoratorA></DecoratorB>
```

And voilà! These decorators allowed us to modify the behavior of `ComponentA` without having an impact on the code. However, our composition root is beginning to get messy as we are instantiating multiple dependencies inside each other. This could make our application harder to maintain. Moreover, the code is becoming harder to read. Furthermore, the code would be even harder to read if the decorators were depending on other classes as well.

As we mentioned previously, you can use decorators to change the behavior or state of an object; be creative. For example, you could create a class that queries remote resources, say over HTTP, and then decorate that class with a small component that manages a memory cache of the results, limiting the round trip to the remote server. You could then create

another decorator that monitors the time needed to query those resources and then log that somewhere. This could be a nice exercise to code if you are looking to practice.

Next, we get rid of the `new` keywords.

Project – Decorator using Scrutor

The objective of this update is to simplify the composition of the system we just created. To achieve this, we use **Scrutor**, an open source library that allows us to do just that, among other things.

The first thing we need to do is install the NuGet package using Visual Studio or the CLI. When using the CLI, run the following command:

```
dotnet add package Scrutor
```

Once Scrutor has been installed, you can use the `Decorate<TService, TDecorator>()` extension method on `IServiceCollection` to add decorators.

By using Scrutor, we can update the following messy line:

```
builder.Services.AddSingleton<IComponent>(serviceProvider => new DecoratorB(new DecoratorA(serviceProvider)));
```

And convert it into these three more elegant lines:

```
builder.Services
    .AddSingleton<IComponent, ComponentA>()
    .Decorate<IComponent, DecoratorA>()
    .Decorate<IComponent, DecoratorB>()
;
```

OK; what happened here?

We registered `ComponentA` as the implementation of `IComponent`, with a singleton lifetime, just like the first time.

Then, by using Scrutor, we told the IoC container to override that first binding and to decorate the already registered `IComponent` (`ComponentA`) with an instance of `DecoratorA` instead. Then, we overrode the second binding by telling the IoC container to return an instance of `DecoratorB` that decorates the last known binding of `IComponent` instead (`DecoratorA`).

The result is the same as what we did previously, but is now written in a more elegant and flexible manner. The IoC container injects the equivalent of the following `instance` with a singleton lifetime:

```
var instance = new DecoratorB(new DecoratorA(new ComponentA()));
```

Note

Why am I saying that it is more elegant and flexible? This is a simple example, but if we start adding other dependencies to those classes, it could quickly end up as a complex code block

that could turn into a maintenance nightmare, become very hard to read, and have manually managed lifetimes. Of course, if the system is simple, you can always instantiate the decorators manually without loading an external library. Using methods to encapsulate the initialization of some part of the system is also an option.

Whenever possible, keep your code simple. Using Scrutor is one way to achieve this. Code simplicity helps in the long run as it is easier to read and follow, even for someone else reading it. Always consider the fact that someone else may maintain your code.

To validate that both programs behave the same, with or without Scrutor, the following integration test runs for both projects and ensures their correctness. See `StartupTest.cs` (<https://adpg.link/Tbeh>):

```
[Fact]
public async Task Should_return_a_double_decorated_string()
{
    // Arrange
    var client = _webApplicationFactory.CreateClient();
    // Act
    var response = await client.GetAsync("/");
    // Assert
    response.EnsureSuccessStatusCode();
    var body = await response.Content.ReadAsStringAsync();
    Assert.Equal(
        "Operation: <DecoratorB><DecoratorA>Hello from ComponentA</DecoratorA></DecoratorB>",
        body
    );
}
```

The preceding test sends an HTTP request to one of the applications running in memory and compares the server response to the expected value. Since both projects should have the same output, that test is reused in both `DecoratorPlainStartupTest` and `DecoratorScrutorStartupTest`.

Scrutor

You can also do assembly scanning using Scrutor (<https://adpg.link/xvFS>), which allows you to perform automatic dependency registration. This is outside the scope of this chapter, but it is worth looking into. Scrutor allows you to use the built-in IoC container for more complex scenarios, postponing the need to replace it with a third-party one.

Conclusion

The Decorator pattern is one of the simplest but most powerful design patterns out there. It augments existing classes without modifying them. Moreover, if you don't need to decorate all instances of `x` by `y`, you can encapsulate small blocks of logic, then create complex and granular object trees to fit different needs; this can even be modified at runtime.

The Decorator pattern helps us stay in line with the **SOLID** principles, as follows:

- **S**: The Decorator pattern suggests creating small classes to add behaviors to other classes, which segregates responsibilities.
- **O**: Decorators add behaviors to other classes without modifying them, which is literally the definition of the OCP.
- **L**: N/A
- **I**: By following the ISP, it should be easy to create decorators for your specific needs. If your interfaces are too complex, packing in too many responsibilities, using decorators could be harder. Having a hard time creating a decorator is a good indicator that something is wrong with the design. A well-segregated interface should be easy to decorate.
- **D**: Depending on abstractions is the key to the Decorator's power.

Next, we explore the Composite pattern, which helps us manage complex objects' structures in a different way than how the decorator does.

Implementing the Composite design pattern

The Composite design pattern is another structural GoF pattern that helps us manage complex object structures.

Goal

The goal behind the Composite pattern is to create a hierarchical data structure where you don't need to differentiate groups of elements from a single element, making the hierarchy easy to use for its consumers.

You could think of the Composite pattern as a way of building a graph or a tree with self-managing nodes.

Design

The design is straightforward; we have *components* and *composites*. Both implement a common interface that defines the shared operations. The *components* are the single nodes, while the *composites* are collections of *components*. Let's take a look at a diagram:

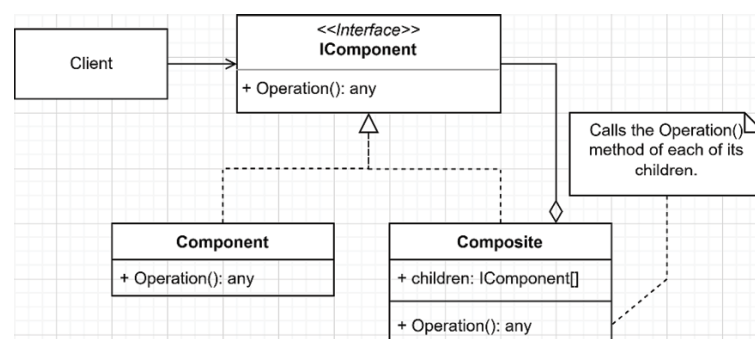


Figure 9.7: Composite class diagram

In the preceding diagram, **Client** depends on an **IComponent** interface. By doing so, it is unaware of the implementation it is using; it could be a **Component** or a **Composite**. Then, we have two implementations:

- `Component` represents a single element; a leaf.
- `Composite` represents a collection of `IComponent`. The `Composite` object uses its children to manage the hierarchy's complexity by delegating part of the process to them.

Those three pieces, when put together, create the Composite design pattern. Considering that it is possible to add `Composite` and `Component` as children of other `Composite` objects, it is possible to create complex, non-linear, and self-managed data structures with next to no effort.

Note

You are not limited to one implementation of `Component` and one implementation of `Composite`; you can create as many implementations of `IComponent` as you need to, based on your use case. Then, you can mix and match them so your data structure fits your needs. We explore how to display complex composites in *Chapter 17, ASP.NET Core User Interfaces*.

Project – BookStore

Let's revisit the bookstore that we built in *Chapter 3, Architectural Principles*.

Context: The store is going so well that our little program is not enough anymore. Our fictional company now owns multiple stores, so they need to divide those stores into sections, and they need to manage book sets and single books. After a few minutes of gathering information, we realize that they can have sets of sets, subsections, and possibly sub-stores, so we need a flexible design.

Let's use the Composite pattern to solve this problem. The user interface that we are aiming to build looks like this:

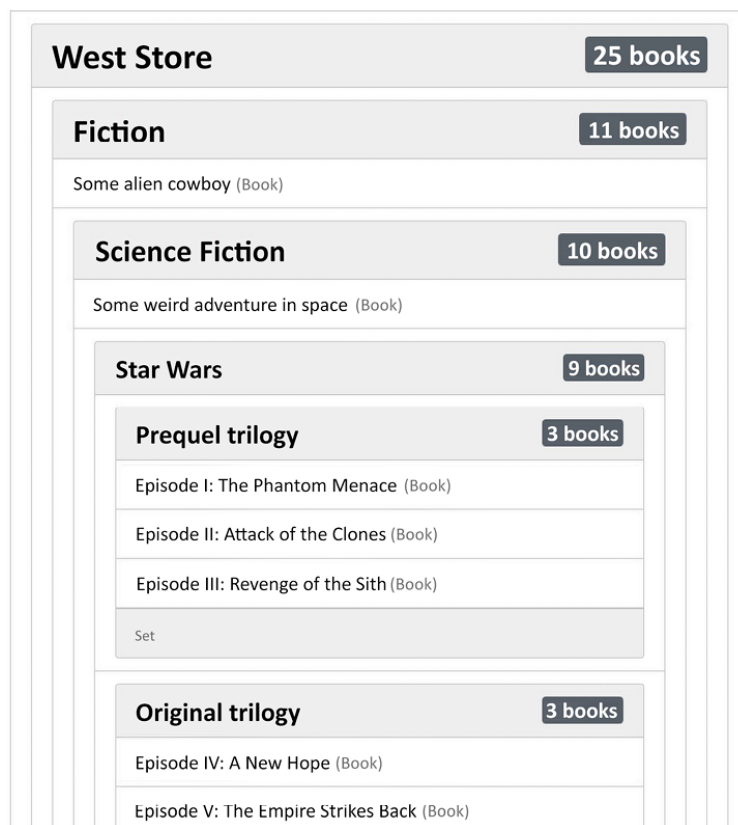


Figure 9.8: The Bookstore project user interface rendered in a browser

First, let's look at the `IComponent` interface, which is the primary building block of the Composite pattern:

```
public interface IComponent
{
    void Add(IComponent bookComponent);
    void Remove(IComponent bookComponent);
    string Display();
    int Count();
    string Type { get; }
}
```

This interface defines the following:

- The `Add()` and `Remove()` sub-components.
- `Display()`, for displaying the current component.
- `Count()`, for counting the number of books available from the current component.
- Knowing the `Type` of the component (displayed in the card's footer).

From there, we need components. First, let's focus on the `BookComposite` class, which abstracts away most of the composite logic:

```
public abstract class BookComposite : IComponent
{
    protected readonly List<IComponent> children = new();
    public BookComposite(string name)
    {
        Name = name ?? throw new ArgumentNullException(nameof(name));
    }
    public string Name { get; }
    public virtual string Type => GetType().Name;
}
```

```

// Not part of the IComponent interface.
// Sets a different heading tag per sub-class,
// see the AppendHeader method.
protected abstract string HeadingTagName { get; }
public virtual void Add(IComponent bookComponent)
{
    children.Add(bookComponent);
}
public virtual int Count()
{
    return children.Sum(child => child.Count());
}
public virtual string Display()
{
    var sb = new StringBuilder();
    sb.Append("<section class=\"card\">");
    AppendHeader(sb);
    AppendBody(sb);
    AppendFooter(sb);
    sb.Append("</section>");
    return sb.ToString();
}
private void AppendHeader(StringBuilder sb)
{
    sb.Append($"<{HeadingTagName} class=\"card-header\">");
    sb.Append(Name);
    sb.Append($"<span class=\"badge badge-secondary float-right\">{Count()} books</span>");
    sb.Append($"</{HeadingTagName}>");
}
private void AppendBody(StringBuilder sb)
{
    sb.Append($"<ul class=\"list-group list-group-flush\">");
    children.ForEach(child =>
    {
        sb.Append($"<li class=\"list-group-item\">");
        sb.Append(child.Display());
        sb.Append("</li>");
    });
    sb.Append("</ul>");
}
private void AppendFooter(StringBuilder sb)
{
    sb.Append("<div class=\"card-footer text-muted\">");
    sb.Append($"<small class=\"text-muted text-right\">{Type}</small>");
    sb.Append("</div>");
}
public virtual void Remove(IComponent bookComponent)
{
    children.Remove(bookComponent);
}
}

```

Note

In this case, to focus on the Composite pattern,

IComponent's implementations handle how its data is presented. However, most of the time, I would not recommend doing so. Why? Because we are giving too many responsibilities to those classes since we are tightly coupling them with the HTML language. It makes the components harder to reuse. Think Single-Responsibility Principle (SRP). We revisit these concepts in subsequent chapters and fix this problem.

The `BookComposite` class implements the following shared features:

- Children management (highlighted in the code).
- Setting the `Name` property of the composite object.
- Automatically finding the `Type` name of its derived class.
- Counting the number of children (and, implicitly, the children's children).
- Displaying the composite and its children.

Note

Using the LINQ `Sum()` extension method in the `children.Sum(child => child.Count());` expression allowed us to replace a more complex `for` loop and an accumulator variable.

Now, let's take a look at a more complex composite example. By creating multiple classes, we can pinpoint what responsibilities we have. In a real scenario, we may need to handle more than a name and a count. Moreover, it shows how flexible the Composite pattern is.

Here is the full hierarchy that represents our bookstore:

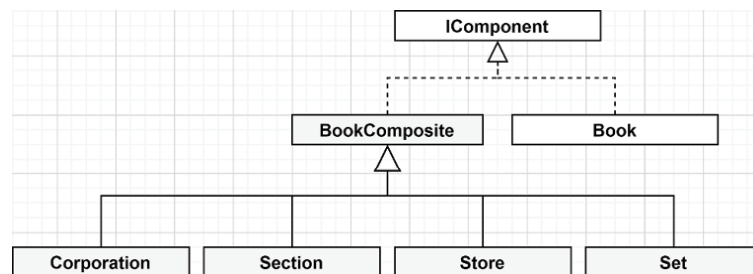


Figure 9.9: Inheritance hierarchy of the BookStore project

Under `BookComposite`, we have the following:

- `Corporation`, which represents the corporation that owns multiple stores. However, it is not limited to owning stores; a corporation could own other corporations and stores, or any other `IComponent` for that matter.
- `Section`, which is a section of a bookstore, or a category of books.
- `Store`, which represents a bookstore.
- `Set`, which is a book set, such as a trilogy.

All of these can be composed of any `IComponent`, making this an ultra-flexible data structure. Before we move on, let's look at the code for these `BookComposite` subclasses:

```
public class Corporation : BookComposite
{
    public Corporation(string name) : base(name) { }
    protected override string HeadingTagName => "h1";
}
public class Store : BookComposite
{
    public Store(string name) : base(name) { }
    protected override string HeadingTagName => "h2";
}
```

```

    }
    public class Section : BookComposite
    {
        public Section(string name) : base(name) { }
        protected override string HeadingTagName => "h3";
    }
    public class Set : BookComposite
    {
        public Set(string name, params IComponent[] books)
            : base(name)
        {
            foreach (var book in books)
            {
                Add(book);
            }
        }
        protected override string HeadingTagName => "h4";
    }
}

```

As you can see, the code is straightforward; the subclasses inherit from `BookComposite`, which does most of the work, leaving them to specify only the value of the `HeadingTagName` property. `Set` is different and allows us to inject other `IComponent` objects into its constructor. This is going to be convenient later when we assemble the tree (Hint: A book set contains multiple books).

The last part of our Composite pattern's implementation is the `Book` class:

```

public class Book : IComponent
{
    public Book(string title)
    {
        Title = title ?? throw new ArgumentNullException(nameof(title));
    }
    public string Title { get; set; }
    public string Type => "Book";
    public int Count() => 1;
    public string Display() => $"{Title} <small class=\"text-muted\">({Type})</small>";
    public void Add(IComponent bookComponent) => throw new NotSupportedException();
    public void Remove(IComponent bookComponent) => throw new NotSupportedException();
}

```

The `Book` class is a little different as it is not a collection of other objects, but a single node. Let's look at the differences:

- It has a `Title` property, instead of `Name`. How to name a component is not defined in the `IComponent` interface, so we can do what we want; in this case, a book has a title, not a name.
- It returns `"Book"` as the value of its `Type` property.
- It tells the callers that both the `Add()` and `Remove()` operations are not supported by throwing an exception.
- Its `Count()` method always returns `1` because an instance of the `Book` class represents a single book and, by extension, only represent themselves: one book; this is the leaf.
- The `Display()` method is also way simpler because it only needs to handle itself; there are no children.

Before we jump into the program, let's look at the last part that was added to help encapsulate the data structure's creation. This is not part of the

Composite pattern, but now that we know what a factory is, we can use one to encapsulate the creation logic of our data structure. The factory interface looks like the following:

```
public interface ICorporationFactory
{
    Corporation Create();
}
```

The default concrete implementation of `ICorporationFactory` is `DefaultCorporationFactory`, and it creates the following structure (visual representation; see <https://adpg.link/2Vqw> for the full-size image):

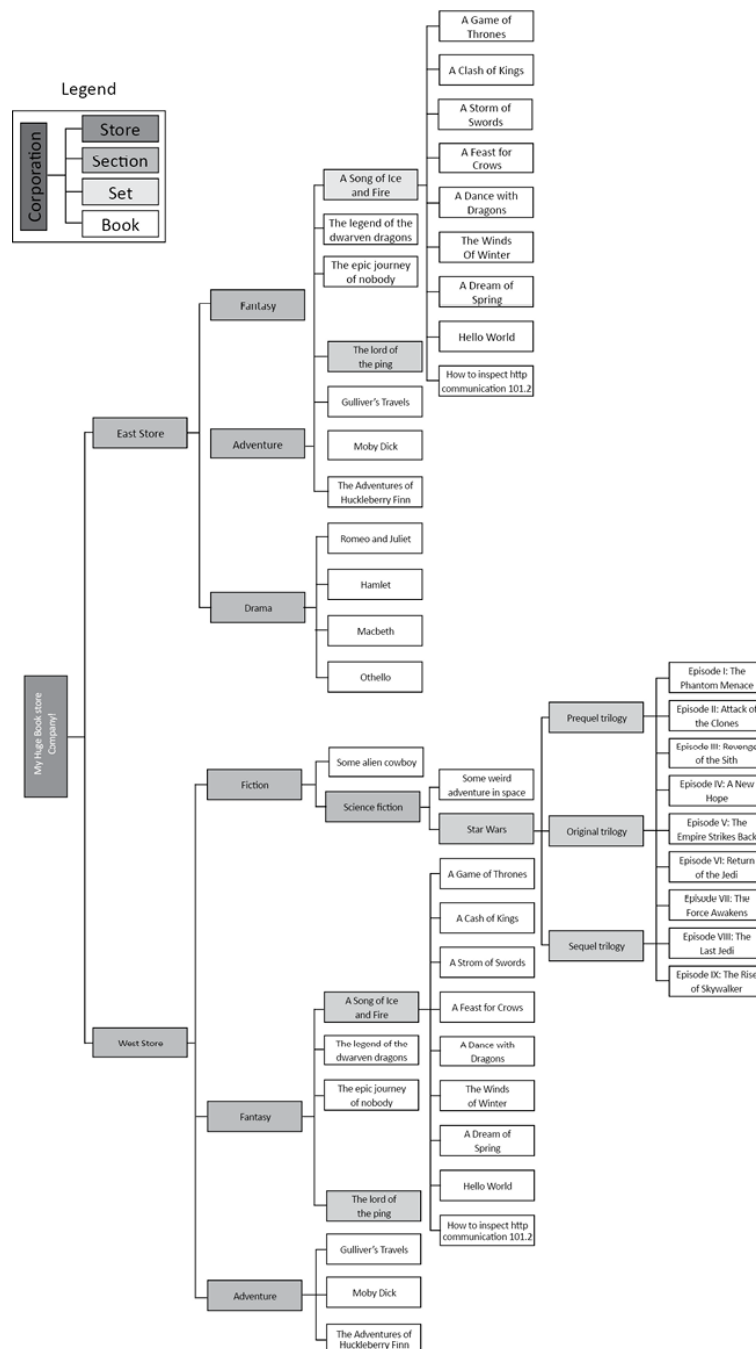


Figure 9.10: A representation of the data that the `DefaultCorporationFactory` class creates

If we take a close look, we can see that the structure is non-linear. There are sections, subsections, sets, and subsets. We could even add books directly to the store if we want. This whole structure is defined using our composite model in `DefaultCorporationFactory`.

To keep it simple, let's focus on the West Store's Fiction section:

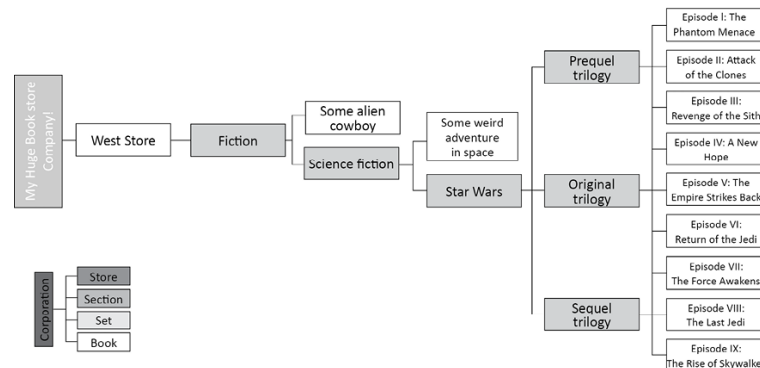


Figure 9.11: The Fiction section of the West Store data

In the West Store, we have a section that contains a fabricated book and a subsection named Science fiction. In Science fiction, there is another fabricated book and a set of books named Star Wars. Three subsets represent the three trilogies under the Star Wars set, which reveal the flexibility of the design.

Let's take a look at the code that builds that section in isolation. Feel free to consult the full source code for the complete example (<https://adpg.link/DD8e>). Here is the code:

```

public class DefaultCorporationFactory : ICorporationFactory
{
    public Corporation Create()
    {
        var corporation = new Corporation("My Huge Book Store Company!");
        corporation.Add(CreateEastStore());
        corporation.Add(CreateWestStore());
        return corporation;
    }
    private IComponent CreateWestStore()
    {
        var store = new Store("West Store");
        store.Add(CreateFictionSection());
        store.Add(CreateFantasySection());
        store.Add(CreateAdventureSection());
        return store;
    }
    private IComponent CreateFictionSection()
    {
        var section = new Section("Fiction");
        section.Add(new Book("Some alien cowboy"));
        section.Add(CreateScienceFictionSection());
        return section;
    }
    private IComponent CreateScienceFictionSection()
    {
        var section = new Section("Science Fiction");
        section.Add(new Book("Some weird adventure in space"));
        section.Add(new Set(

```

```

        "Star Wars",
        new Set(
            "Prequel trilogy",
            new Book("Episode I: The Phantom Menace"),
            new Book("Episode II: Attack of the Clones"),
            new Book("Episode III: Revenge of the Sith")
        ),
        new Set(
            "Original trilogy",
            new Book("Episode IV: A New Hope"),
            new Book("Episode V: The Empire Strikes Back"),
            new Book("Episode VI: Return of the Jedi")
        ),
        new Set(
            "Sequel trilogy",
            new Book("Episode VII: The Force Awakens"),
            new Book("Episode VIII: The Last Jedi"),
            new Book("Episode IX: The Rise of Skywalker")
        )
    ));
    return section;
}
// ...
}

```

I find the preceding code very exhaustive, making the creation of this part of the data structure clear. Now that we've read part of the code of the factory, let's head back to the Composite pattern and learn how to display it. In short, we only need to call the `Display()` method of the root node of our composite model (highlighted), like this:

```

using Composite.Services;
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<ICorporationFactory, DefaultCorporationFactory>();
var app = builder.Build();
app.MapGet("/", async (HttpContext context, ICorporationFactory corporationFactory) =>
{
    var compositeDataStructure = corporationFactory.Create();
    var output = compositeDataStructure.Display();
    context.Response.Headers.Add("Content-Type", "text/html; charset=utf-8");
    await context.Response.WriteAsync("[HTML removed for brevity]");
    await context.Response.WriteAsync(output);
    await context.Response.WriteAsync("[HTML removed for brevity]");
});
app.Run();

```

Let's take a look at what's happening in the preceding code:

- We are telling the IoC container to bind the `DefaultCorporationFactory` class to the `ICorporationFactory` interface.
- In the minimal API delegate, we inject an implementation of `ICorporationFactory`, build the data structure, and display it. This is the part that represents the consumer of our Composite pattern implementation.

Now, let's analyze the content of the `MapGet` delegate in more detail:

1. We use the injected `ICorporationFactory corporationFactory` parameter to create the data structure.

2. We call the `Display()` method to generate the output; the highlighted line. This is where the composite magic happens.
3. Finally, we write that output to the response stream by calling `await context.Response.WriteAsync(output);`. We wrap the content of the `output` variable with omitted HTML that adds the default tags like `<html>`, `<head>`, and `<body>`, registers Bootstrap, and so on.

The Composite pattern allowed us to render a complex data structure in a small method call. Since each component handles itself in an autonomous fashion, the burden of handling this complexity is taken away from the consumer.

In another scenario, we could have used the data instead of blindly displaying it; we could have also implemented a way of browsing that data or any other use cases that may come to mind. In this code example, I added a bit of complexity on purpose so that we could experiment with the Composite pattern in a more complex scenario while keeping extraneous details away from the code as much as possible.

I encourage you to play around with the existing data structure so that you understand the pattern. You could also try adding a `Movie` class to manage movies; a bookstore must diversify its activities. You could also differentiate movies from books so that customers are not confused. The bookstores could have physical and digital books as well.

If, after all of that, you are still looking for more, try building a new application from scratch and using the Composite pattern to create, manage, and display a multi-level menu.

Conclusion

The Composite pattern is very effective at building, managing, and maintaining complex non-linear data structures. Its power is primarily in its self-management capabilities. Each node, component, or composite is responsible for its own logic, leaving little to no work for the composite's consumers.

Using the Composite pattern helps us follow the **SOLID** principles in the following ways:

- **S**: It helps divide multiple elements of a complex data structure into small classes, in order to split responsibilities.
- **O**: By allowing us to “mix and match” different implementations of `IComponent`, the Composite pattern allows us to extend the data structure without impacting the other existing classes. For example, you could create a new class that implements `IComponent` and start using it right away, with no need to modify any other component classes.
- **L**: N/A
- **I**: The Composite pattern may violate the ISP when single items implement operations that only impact the collections.
- **D**: All of the Composite pattern actors depend solely on `IComponent`.

Next, we move to a different type of structural pattern that adapts one interface to another.

Implementing the Adapter design pattern

The Adapter pattern is another structural GoF pattern that helps adapt the API of one class to the API of another interface.

Goal

The adapter's goal is to plug in a component that does not respect the expected contract and adapt it so that it does. The adapter comes in handy when you cannot change the adaptee's code or if you do not want to change it.

Design

Think of the adapter as a power outlet's universal adapter; you can connect a North American device to a European outlet by connecting it to the adapter and then connecting it to the power outlet. The Adapter design pattern does exactly that, but for APIs.

Let's start by looking at the following diagram:

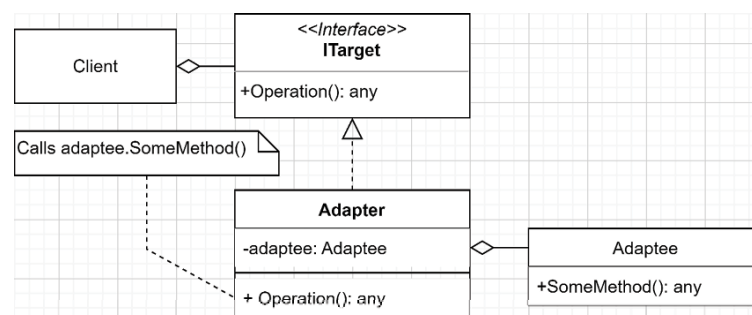


Figure 9.12: Adapter class diagram

In the preceding diagram, we have the following actors:

- **ITarget** , which is the interface that holds the contract that we want (or have) to use.
- **Adaptee** , which is the concrete component that we want to use that does not conform to **ITarget**.
- **Adapter** , which adapts the **Adaptee** class to the **ITarget** interface.

There is a second way of implementing the Adapter pattern that implies inheritance. If you can go for composition, go for it, but if you need access to protected methods or other internal states of **Adaptee** , you can go for inheritance instead, like this:

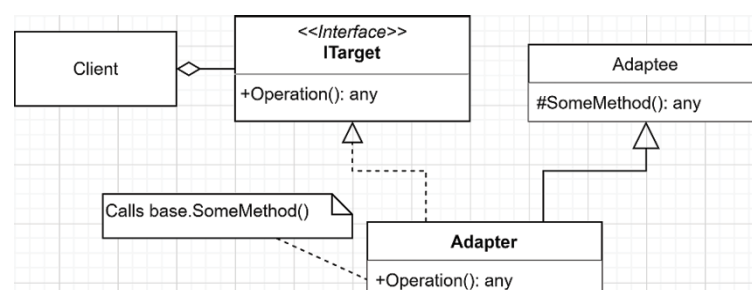


Figure 9.13: Adapter class diagram inheriting the Adaptee

The actors are the same, but instead of composing `Adapter` with `Adaptee`, `Adapter` inherits from `Adaptee`. This makes `Adapter` become both an `Adaptee` and an `ITarget`.

Project – Greeter

Context: We've programmed a highly sophisticated greeting system that we want to reuse in a new program. However, its interface does not match the new design, and we cannot modify it because other systems use that greeting system.

To fix this problem, we decided to apply the Adapter pattern. Here is the code of the external greeter (`ExternalGreeter`), and the new interface (`IGreeter`) used in the new system. This code must not directly modify the `ExternalGreeter` class to prevent any breaking changes from occurring in other systems:

```
public interface IGreeter
{
    string Greeting();
}
public class ExternalGreeter
{
    public string GreetByName(string name)
    {
        return $"Adaptee says: hi {name}!";
    }
}
```

Next is how the external greeter is adapted to meet the latest requirements:

```
public class ExternalGreeterAdapter : IGreeter
{
    private readonly ExternalGreeter _adaptee;
    public ExternalGreeterAdapter(ExternalGreeter adaptee)
    {
        _adaptee = adaptee ?? throw new ArgumentNullException(nameof(adaptee));
    }
    public string Greeting()
    {
        return _adaptee.GreetByName("ExternalGreeterAdapter");
    }
}
```

In the preceding code, the actors are as follows:

- `IGreeter`, which represents `ITarget`. This is the interface that we want to use.
- `ExternalGreeter`, which represents `Adaptee`. This is the external component that already contains all the logic that's been programmed and tested. This could be located in an external assembly, maybe even installed from NuGet.
- `ExternalGreeterAdapter`, which represents `Adapter`. This is where the adapter's job is done:
`ExternalGreeterAdapter.Greeting()` calls

```
ExternalGreeter.GreetByName("ExternalGreeterAdapter"),
```

which implements the greeting logic.

Now, we can call the `IGreeter.Greeting()` method and get the result of the

`ExternalGreeter.GreetByName("ExternalGreeterAdapter")` call.

With this in place, we can reuse the existing logic. We can test any

`IGreeter` consumers by mocking the `IGreeter` interface, without caring about the `ExternalGreeterAdapter` class.

I have to admit that the “complex logic” in this case is pretty simple, but we are here for the Adapter pattern, not for imaginary business logic.

Now, let’s take a look at the consumer:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddSingleton<ExternalGreeter>();
builder.Services.AddSingleton<IGreeter, ExternalGreeterAdapter>();
var app = builder.Build();
app.MapGet("/", (IGreeter greeter) => greeter.Greeting());
app.Run();
```

In the preceding code, we composed our application by specifying that the same instance of `ExternalGreeterAdapter` should be provided every time we ask for an `IGreeter` interface (singleton). We also told the container to provide a single instance of `ExternalGreeter` whenever it’s requested (in this case, it’s injected into `ExternalGreeterAdapter`).

Then, the consumer (**Client** in the preceding class diagrams) is the `app.MapGet("/", ...)` delegate (see the highlighted code of the preceding code block). `IGreeter` is injected as a parameter of the delegate when an HTTP call is made to `/`. Then, the delegate calls the `Greeting` method on that injected instance. Finally, it outputs the greeting to the response stream automatically.

The following diagram represents what’s happening in this system:

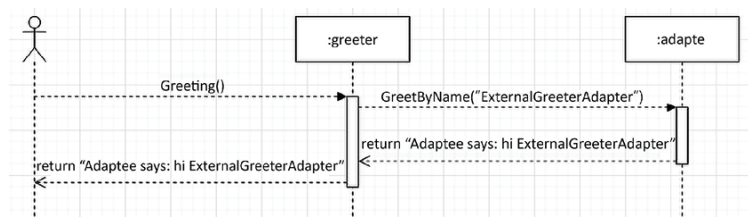


Figure 9.14: Greeter system sequence diagram

And voilà! We’ve adapted the `ExternalGreeterAdapter` class to the `IGreeter` interface with little effort.

Conclusion

The Adapter pattern is another simple pattern that offers flexibility. With it, we can use older or non-conforming components without rewriting them. Of course, depending on the `ITarget` and `Adaptee` interfaces, you may need to put more or less effort into writing the code of the `Adapter` class.

Now, let's learn how the Adapter pattern can help us follow the **SOLID** principles:

- **S:** The Adapter pattern has only one responsibility: make an interface work with another interface.
- **O:** The Adapter pattern allows us to modify the Adaptee's interface without the need to modify its code.
- **L:** Inheritance is not much of a concern when it comes to the Adapter pattern, so once again, this principle does not apply. If `Adapter` inherits from `Adaptee`, the goal is to change its interface, not its behavior, which should conform to the LSP.
- **I:** Since we could see the `Adapter` class as a means to an end where `ITarget` is the end, the Adapter pattern is directly depending on the design of `ITarget` and has no direct impact on it (good or bad). Your only concern for this principle is to design `ITarget` well enough so that it follows the ISP, which is not part of the pattern, but the reason to use the pattern.
- **D:** The Adapter pattern introduces only an implementation of the `ITarget` interface. Even if the adapter depends on a concrete class, its goal is to break the direct dependency on that external component by adapting it to the `ITarget` interface.

Next, we explore the last structural pattern of the chapter that teaches foundational concepts.

Implementing the Façade design pattern

The Façade pattern is another structural GoF pattern, similar to the Adapter pattern. It creates a wall (a façade) between one or more subsystems. The big difference between the adapter and the façade is that instead of adapting an interface to another, the façade simplifies the use of a subsystem, typically by using multiple classes of that subsystem.

Note

The same idea can be applied to shielding one or more programs, but in this case, the façade is called a gateway—more on that in *Chapter 16, Introduction to Microservices Architecture*.

The Façade pattern is an extremely useful pattern that can be adapted to multiple situations.

Goal

The goal of the Façade pattern is to simplify the use of one or more subsystems by providing an interface that is easier to use than the subsystems themselves, shielding the consumers from that complexity.

Design

We could create multiple diagrams representing a multitude of subsystems, but let's keep things simple here. Remember that you can replace the single subsystem shown in the following diagram with as many subsystems as you need to adapt:

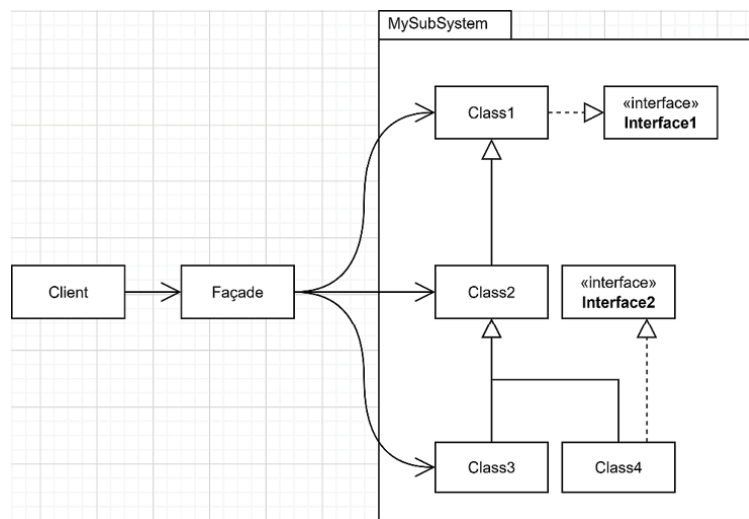


Figure 9.15: A class diagram representing a *Façade* object that hides a complex subsystem

As we can see, **Façade** plays the intermediary between **Client** and the subsystem, simplifying its usage. Let's see this in action as a sequence diagram:

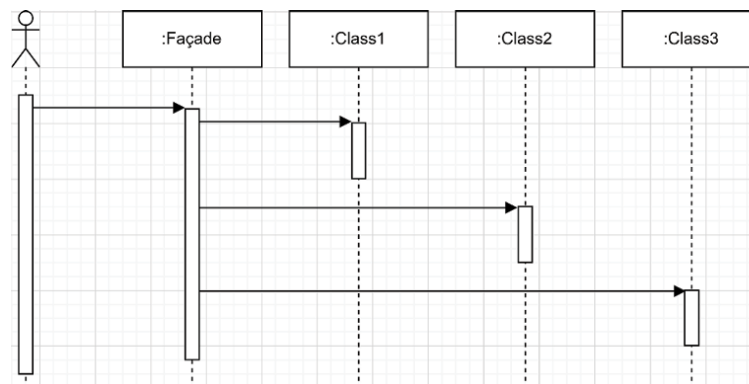


Figure 9.16: A sequence diagram representing a *Façade* object that interacts with a complex subsystem

In the preceding diagram, **Client** calls **Façade** once, while **Façade** places multiple calls against different classes.

There are multiple ways of implementing a façade:

- **Opaque façades:** In this form, the `Façade` class is inside the subsystem. All other classes of the subsystem have an `internal` visibility modifier. This way, only the classes inside the subsystem can interact with the other internal classes, forcing the consumers to use the `Façade` class.
- **Transparent façades:** In this form, the classes can have a `public` modifier, allowing the consumers to use them directly or to use the `Façade` class. This way, we can create the `Façade` class inside or outside the subsystem.
- **Static façades:** In this form, the `Façade` class is `static`. We can implement a static façade as opaque or transparent. Use this approach only as a last resort because global (`static`) elements tend to limit flexibility and testability.

Project – The façades

In this example, we will play with three C# projects:

- An ASP.NET Core application leveraging minimal APIs to expose four HTTP endpoints. Two endpoints aim at `OpaqueFacadeSubSystem`, while the other two endpoints aim at `TransparentFacadeSubSystem`. This is our consumer.
- The `OpaqueFacadeSubSystem` class library implements an **opaque façade**.
- The `TransparentFacadeSubSystem` class library implements a **transparent façade**.

Let's start with the class libraries.

Note

To follow the SOLID principles, adding some interfaces representing the elements of the subsystem seemed appropriate. In subsequent chapters, we explore how to organize our abstractions to be more reusable, but for now, both abstractions and implementations are in the same assembly.

Opaque façade

In this assembly, only the façade is public; all the other classes are internal, which means they are hidden from the external world. In most cases, this is not ideal; hiding everything makes the subsystem less flexible and harder to extend.

However, in some scenarios, you may want to control access to your internal APIs. This may be because they are not mature enough and you don't want any third party to depend on them, or for any other reasons you may deem appropriate for your specific use case.

Let's start by taking a look at the following subsystem code:

```
// An added interface for flexibility
public interface IOpaqueFacade
{
    string ExecuteOperationA();
    string ExecuteOperationB();
}
// A hidden component
internal class ComponentA
{
    public string OperationA() => "Component A, Operation A";
    public string OperationB() => "Component A, Operation B";
}
// A hidden component
internal class ComponentB
{
    public string OperationC() => "Component B, Operation C";
    public string OperationD() => "Component B, Operation D";
}
// A hidden component
internal class ComponentC
{
    public string OperationE() => "Component C, Operation E";
}
```

```

        public string OperationF() => "Component C, Operation F";
    }
    // The opaque façade using the other hidden components
    public class OpaqueFacade : IOpaqueFacade
    {
        private readonly ComponentA _componentA;
        private readonly ComponentB _componentB;
        private readonly ComponentC _componentC;
        // Using constructor injection
        internal OpaqueFacade(ComponentA componentA, ComponentB componentB, ComponentC componentC)
        {
            _componentA = componentA ?? throw new ArgumentNullException(nameof(componentA));
            _componentB = componentB ?? throw new ArgumentNullException(nameof(componentB));
            _componentC = componentC ?? throw new ArgumentNullException(nameof(componentC));
        }
        public string ExecuteOperationA()
        {
            return new StringBuilder()
                .AppendLine(_componentA.OperationA())
                .AppendLine(_componentA.OperationB())
                .AppendLine(_componentB.OperationD())
                .AppendLine(_componentC.OperationE())
                .ToString();
        }
        public string ExecuteOperationB()
        {
            return new StringBuilder()
                .AppendLine(_componentB.OperationC())
                .AppendLine(_componentB.OperationD())
                .AppendLine(_componentC.OperationF())
                .ToString();
        }
    }
}

```

As you can see, the `OpaqueFacade` class is coupled with `ComponentA`, `ComponentB`, and `ComponentC` directly. There was no point in extracting any internal interfaces since the subsystem is not extensible anyway. We could have done this to offer some kind of internal flexibility, but in this case, there was no advantage in doing so.

Besides this coupling, `ComponentA`, `ComponentB`, and `ComponentC` define two methods each, which returns a string describing their source. With that code in place, we can observe what is happening and how the final result was composed.

`OpaqueFacade` also exposes two methods, but each composes a different message by using the underlying subsystem's components. This is a classic use of a façade; the façade queries other objects in a more or less complicated way and then does something with the results, taking away the caller's burden of knowing the subsystem.

Moreover, to register the `OpaqueFacadeSubSystem` façade against the IoC container, we needed some “magic” to overcome the internal visibility modifiers. To solve this problem, I added the following extension method that registers the dependencies, as we explored in *Chapter 7, Deep Dive into Dependency Injection*. The extension method is accessible by the consumer application:

```

public static class StartupExtensions
{
    public static IServiceCollection AddOpaqueFacadeSubSystem(this IServiceCollection

```

```

    {
        services.AddSingleton<IOpaqueFacade>(serviceProvider
            => new OpaqueFacade(new ComponentA(), new ComponentB(), new ComponentC()))
        return services;
    }
}

```

Next, to the transparent façade implementation.

Transparent façade

The transparent façade is the most flexible type of façade, extremely suitable for a system that leverages dependency injection. The implementation is very similar to the opaque façade, but the `public` visibility modifier changes how consumers can access the class library elements. For this system, it was worth adding interfaces to allow the consumers of the subsystem to extend it when needed.

First, let's take a look at the abstractions:

```

namespace TransparentFacadeSubSystem.Abstractions
{
    public interface ITransparentFacade
    {
        string ExecuteOperationA();
        string ExecuteOperationB();
    }
    public interface IComponentA
    {
        string OperationA();
        string OperationB();
    }
    public interface IComponentB
    {
        string OperationC();
        string OperationD();
    }
    public interface IComponentC
    {
        string OperationE();
        string OperationF();
    }
}

```

The API of this subsystem is the same as the opaque façade. The only difference is how we can use the subsystem and extend it (from a consumer standpoint). The implementations are mostly the same as well, but the classes implement the interfaces and are `public`; the highlighted elements represent the changes that have been made:

```

namespace TransparentFacadeSubSystem
{
    public class ComponentA : IComponentA
    {
        public string OperationA() => "Component A, Operation A";
        public string OperationB() => "Component A, Operation B";
    }

    public class ComponentB : IComponentB
    {
        public string OperationC() => "Component B, Operation C";
        public string OperationD() => "Component B, Operation D";
    }
}

```

```

    }
    public class ComponentC : IComponentC
    {
        public string OperationE() => "Component C, Operation E";
        public string OperationF() => "Component C, Operation F";
    }
    public class TransparentFacade : ITransparentFacade
    {
        private readonly IComponentA _componentA;
        private readonly IComponentB _componentB;
        private readonly IComponentC _componentC;

        public TransparentFacade(IComponentA componentA, IComponentB
componentB, IComponentC componentC)
        {
            _componentA = componentA ?? throw new ArgumentNullException(nameof(componentA)
            _componentB = componentB ?? throw new ArgumentNullException(nameof(componentB)
            _componentC = componentC ?? throw new ArgumentNullException(nameof(componentC)

        }

        public string ExecuteOperationA()
        {
            return new StringBuilder()
                .AppendLine(_componentA.OperationA())
                .AppendLine(_componentA.OperationB())
                .AppendLine(_componentB.OperationD())
                .AppendLine(_componentC.OperationE())
                .ToString();
        }

        public string ExecuteOperationB()
        {
            return new StringBuilder()
                .AppendLine(_componentB.OperationC())
                .AppendLine(_componentB.OperationD())
                .AppendLine(_componentC.OperationF())
                .ToString();
        }
    }
}

```

Before going further, note that the following extension method was also created to simplify the use of the subsystem. However, everything that's defined there can be overridden (which is not the case for the opaque façade):

```

public static class StartupExtensions
{
    public static IServiceCollection AddTransparentFacadeSubSystem(this IServiceCollection
    {
        services.AddSingleton<ITransparentFacade, TransparentFacade>();
        services.AddSingleton<IComponentA, ComponentA>();
        services.AddSingleton<IComponentB, ComponentB>();
        services.AddSingleton<IComponentC, ComponentC>();
        return services;
    }
}

```

As you can see, all the new elements are gone and have been replaced by simple dependency registration (singleton lifetimes, in this case). These little differences give you the tools to reimplement any part of the subsystem if you want to; see the following subsection for an example.

In the transparent façade code, we can register bindings because classes and interfaces are `public`. In the opaque façade, we had to define the constructor of the `OpaqueFacade` class as `internal` because the type of its parameters (`ComponentA`, `ComponentB`, and `ComponentC`) are internal. Changing the visibility modifier of the constructor from `internal` to `public` yields a *CS0051 Inconsistent accessibility* error.

Besides those differences, the transparent façade plays the same role as the opaque façade, outputting the same result.

The program

Now, let's analyze the consumer that is a micro-ASP.NET Core application that forwards HTTP requests to the façades.

The first step is to register the dependencies (in the composition root), like this:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services
    .AddOpaqueFacadeSubSystem()
    .AddTransparentFacadeSubSystem();
```

Note

With these extension methods, the application root is so clean that it is hard to know that we registered two subsystems against the IoC container. This is a good way of keeping your code organized and clean, especially when you're building class libraries.

Now that everything has been registered, the second thing we need to do is route those HTTP requests to the façades. Let's take a look at the code first:

```
var app = builder.Build();
app.MapGet("/opaque/a", (IOpaqueFacade opaqueFacade) => opaqueFacade.ExecuteOperation1);
app.MapGet("/opaque/b", (IOpaqueFacade opaqueFacade) => opaqueFacade.ExecuteOperation1);
app.MapGet("/transparent/a", (ITransparentFacade transparentFacade) => transparentFacade.ExecuteOperation1);
app.MapGet("/transparent/b", (ITransparentFacade transparentFacade) => transparentFacade.ExecuteOperation1);
app.Run();
```

In the preceding block (see highlighted code), we define four routes. Each route dispatches the request to one of the façade's methods. A façade is injected in each delegate, and thanks to the minimal API feature, this code is indeed minimal and very clean.

If you run the program and navigate to `https://localhost:9004/transparent/a`, the page should display the following:

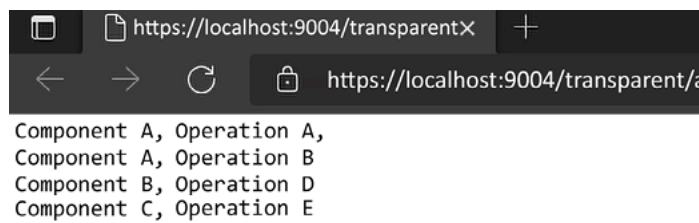


Figure 9.17: The result of executing the `ExecuteOperationA` method of the `ITransparentFacade` instance

What happened is located inside the delegates. It uses the injected `ITransparentFacade` service and calls its `ExecuteOperationA()` method, and then outputs the `result` variable to the response stream.

Now, let's define how `ITransparentFacade` is composed:

- `ITransparentFacade` is an instance of `TransparentFacade`.
- We inject `IComponentA`, `IComponentB`, and `IComponentC` in the `TransparentFacade` class.
- These dependencies are instances of `ComponentA`, `ComponentB`, and `ComponentC`, respectively.

Visually, the following flow happens:

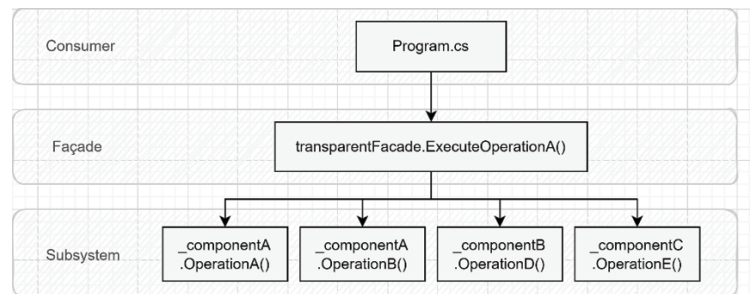


Figure 9.18: A representation of the call hierarchy that occurs when the consumer executes the `ExecuteOperationA` method

In the preceding diagram, we can see the shielding that's done by the façade and how it has made the consumer's life easier. Here, there's one call instead of four.

Note

One of the hardest parts of using dependency injection is its abstractness. If you are not sure how all those parts are assembled, add a breakpoint into Visual Studio (let's say, on the `var result = transparentFacade.ExecuteOperationA()` line) and run the application in debug mode. From there, **Step Into** each method call. That should help you figure out what is happening. Using the debugger to find the concrete types and their states can help find details about a system or diagnose bugs.

To use **Step Into**, you can use the following button or hit **F11**:

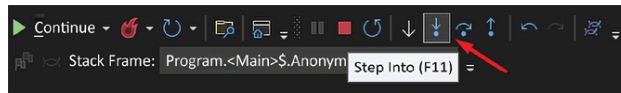


Figure 9.19: The Visual Studio Step Into (F11) button

Next, we update the result without changing the component's code.

Flexibility in action

Now, let's see the added flexibility of the transparent façade in action.

Context: We want to change the behavior of the `TransparentFacade` class. At the moment, the result of the `transparent/b` endpoint looks like this:

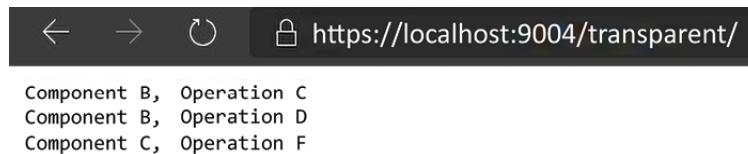


Figure 9.20: The result of executing the `ExecuteOperationB` method of the `ITransparentFacade` instance

To demonstrate that we can extend and change the hidden sub-system (transparent façade), we want to change the output to the following:



Figure 9.21: The expected result once the change has been made

We also want to achieve this result without modifying `ComponentB`. To do this, we execute the following steps:

1. Create the following class:

```
public class UpdatedComponentB : IComponentB
{
    public string OperationC() => "Flexibility";
    public string OperationD() => "Design Pattern";
}
```

2. Tell the IoC container about it, like this:

```
services
    .AddRouting()
    .AddOpaqueFacadeSubSystem()
    .AddTransparentFacadeSubSystem()
    .AddSingleton<IComponentB, UpdatedComponentB>()
;
```

3. From there, if you run the program, you should see the desired result!

Note

Adding a dependency for a second time makes that dependency resolved by the container, thus overriding the first one. However, both registrations remain present in the services collection; for example, calling `GetServices<IComponentB>()` on `IServiceProvider` would return two dependencies. Do not confuse the `GetServices()` and `GetService()` methods (plural versus singular); one returns a collection while the other returns a single instance. That single instance is always the last that has been registered.

That's it! We updated the system without modifying it. This is what dependency injection can do for you when you're designing a program around it.

Alternative façade patterns

One alternative would be to create a *hybrid between a transparent façade and an opaque façade* by exposing the abstractions using the `public` visibility modifier (all of the interfaces) while keeping the implementations hidden under an `internal` visibility modifier. This hybrid design offers the right balance between **control and flexibility**.

Another alternative would be to create a *façade outside of the subsystem*. In the previous examples, we created the façades inside the class libraries, but this is not mandatory; the façade is just a class that creates an accessible wall between your system and one or more subsystems. It should be located wherever you see fit. Creating external façades like this would be especially useful when you do not control the source code of the subsystem(s), such as if you only have access to the binaries. This could also be used to create project-specific façades over the same subsystem, giving you extra flexibility without cluttering your subsystems with multiple façades, shifting the maintenance cost from the subsystems to the client applications that use them.

This one is more of a note than an alternative: you do not need to create an assembly per subsystem. I did it because it helped me explain different concepts to you in the examples, but you could create multiple subsystems in the same assembly. You could even create a single assembly that includes all your subsystems, façades, and the client code (all in a single project).

Conclusion

The Façade pattern is handy for simplifying consumers' lives as it allows you to hide subsystems' implementation details behind a wall. There are multiple flavors to it; the two most prominent ones are:

- The **transparent façade**, which increases flexibility by exposing at least part of the subsystem(s)
- The **opaque façade**, which controls access by hiding most of the subsystem(s)

Now, let's see how the **transparent façade** pattern can help us follow the **SOLID** principles:

- **S:** A well-designed **transparent façade** serves this exact purpose by providing a cohesive set of functionalities to its clients by hiding overly complex subsystems or internal implementation details
- **O:** A well-designed **transparent façade** and its underlying subsystem's components can be extended without direct modification, as we saw in the *Flexibility in action* section
- **L:** N/A
- **I:** By exposing a façade that uses different smaller objects implementing small interfaces, we could say that the segregation is done at both the façade and the component levels
- **D:** The Façade pattern does not specify anything about interfaces, so it is up to the developers to enforce this principle by using other patterns, principles, and best practices

Finally, let's see how the **opaque façade** pattern can help us follow the **SOLID** principles:

- **S:** A well-designed **opaque façade** serves this exact purpose by providing a cohesive set of functionalities to its clients by hiding overly complex subsystems or internal implementation details.
- **O:** By hiding the subsystem, the **opaque façade** limits our ability to extend it. However, we could implement a **hybrid façade** to help with that.
- **L:** N/A
- **I:** The **opaque façade** does not help nor diminish our ability to apply the ISP.
- **D:** The Façade pattern does not specify anything about interfaces, so it is up to the developers to enforce this principle by using other patterns, principles, and best practices.

Summary

In this chapter, we covered multiple fundamental GoF structural design patterns. They help us extend our systems from the outside, without modifying the actual classes, leading to a higher degree of cohesion by composing our object graph dynamically.

We started with the Decorator pattern, which extends other objects, at runtime, by using them internally. Decorators can also be chained, allowing even greater flexibility (decorating other decorators). We also used an open source tool named Scrutor that simplifies the decorator pattern usage by extending the built-in ASP.NET Core dependency injection system.

Then, we covered the Composite pattern, which allows us to create complex and flexible data structures. To make the life of its consumer easier, the composite delegates the navigation responsibility to each component.

After that, we covered the Adapter pattern, which allows us to adapt an object to another interface. This pattern is very helpful when we need to adapt the components of external systems that we have no control over.

Finally, we dug into the Façade pattern, which is similar to the Adapter pattern, but at the subsystem level. It allows us to create a wall in front of one or more subsystems, simplifying its usage. It could also be used to hide a subsystem from its consumers.

In the next chapter, we explore two GoF behavioral design patterns: the Template method and the Chain of Responsibility design pattern.

Questions

Here are a few revision questions:

1. Can we decorate a decorator with another decorator?
2. Name one of the advantages of the Composite pattern.
3. Can we use the Adapter pattern to migrate an old API to a new system in order to adapt its APIs before rewriting it?
4. Why should we use a façade?
5. What is the difference between the Adapter and the Façade patterns?

Further reading

- To learn more about Scrutor, please visit <https://adpg.link/xvfS>

Join our book's Discord space

Join the book's Discord workspace for *Ask me Anything* session with the authors:

<https://packt.link/ASPdotNET6DesignPatterns>



[Support](#) [Sign Out](#)

©2022 O'REILLY MEDIA, INC. [TERMS OF SERVICE](#) [PRIVACY POLICY](#)